Project Title: NEXUS AI – Intelligent Research Workspace & Admin Control

(Basic HTML, Hyperlink and Commenting code) [Font Cambria size 12]

1. **JS Validation:**

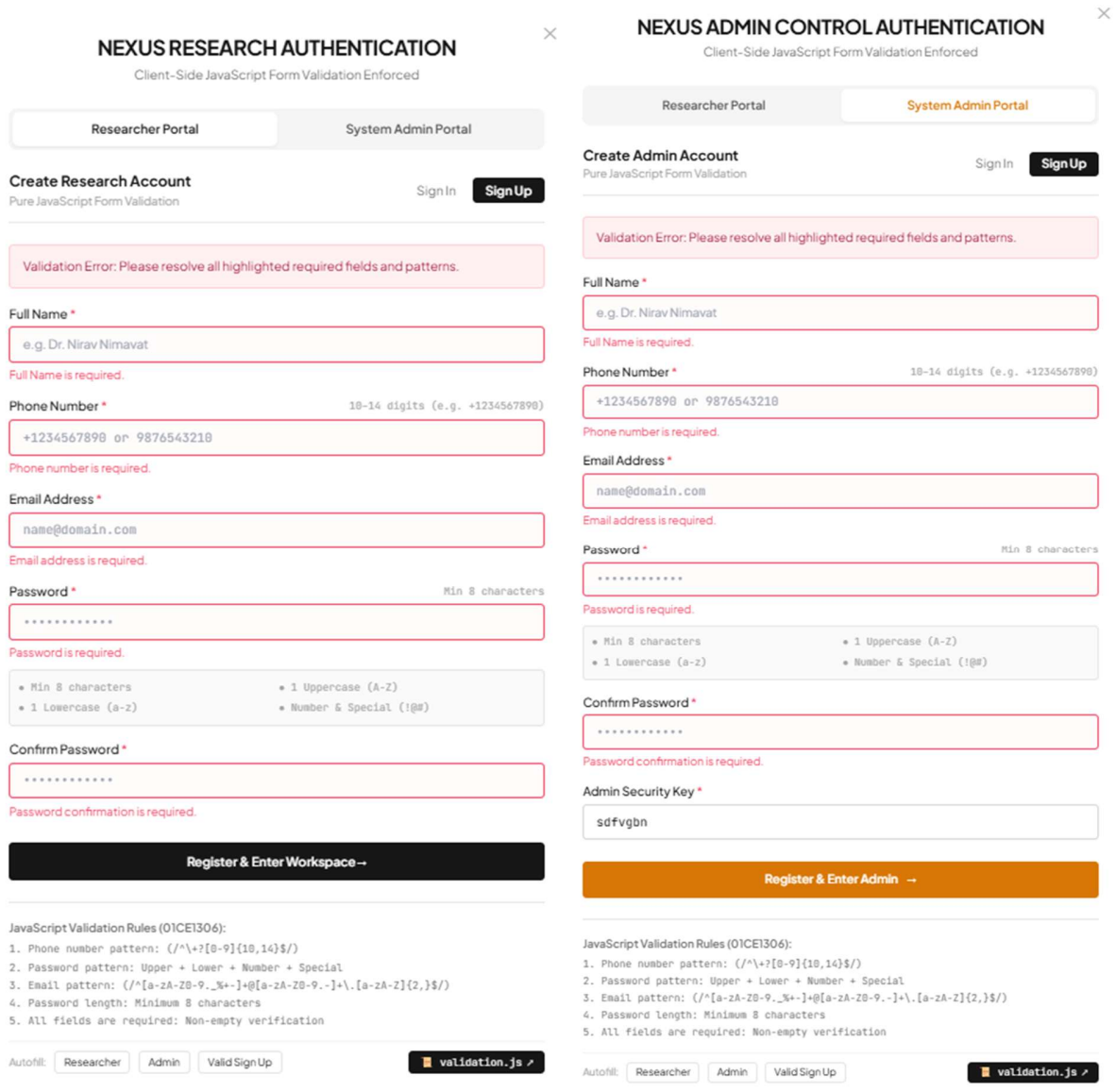


Fig 1.1 JS Validation: All fields are required



Fig 1.2 JS Validation: Password Length



Fig 1.3 JS Validation: Email Pattern



Fig 1.4 JS Validation: Password Pattern

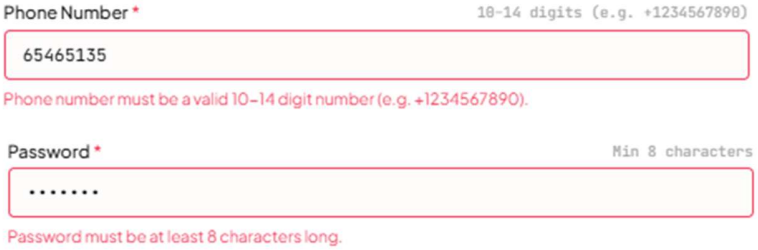


Fig 1.5 JS Validation: Phone number pattern

Source Code:

```
/**
 *
==========================================================================
```

```
========
 * NEXUS AI – Intelligent Research Workspace & Admin Control
 * Course: Web Technology (01CE1306)
 * Student Name: Nirav Nimavat (Enrollment No: 92610103001)
 *
 * FILE: validation.js
 * PURPOSE: Pure JavaScript Form Validation for Dual-Sided Authentication
 *        (Researcher Portal & Admin Control Portal)
 *
================================================================================
========
 *
 * THE 5 MANDATORY JAVASCRIPT VALIDATIONS IMPLEMENTED IN THIS FILE:
 *  1. Phone number pattern (/^\+?[0-9]{10,14}$/)
 *  2. Password pattern (At least 1 Uppercase, 1 Lowercase, 1 Number, 1 Special Char)
 *  3. Email pattern (/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/)
 *  4. Password length (Minimum 8 characters)
 *  5. All fields are required (Non-empty verification)
 *
================================================================================
========
 */

//
================================================================================
========
// 1. REGEX PATTERNS DEFINITION
//
================================================================================
========
const VALIDATION_PATTERNS = {
  // Rule 1: 10 to 14 digits, optional leading '+' for international format
  PHONE: /^\+?[0-9]{10,14}$/,

  // Rule 2: Pattern components for strong password security
  PASSWORD_UPPERCASE: /[A-Z]/,
  PASSWORD_LOWERCASE: /[a-z]/,
  PASSWORD_NUMBER: /[0-9]/,
  PASSWORD_SPECIAL: /[!@#$%^&*(),.?":{}|<>_~-]/,

  // Rule 3: Standard RFC compliant email regex pattern
```

```
  EMAIL: /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/,

  // Rule 4: Minimum length threshold
  PASSWORD_MIN_LENGTH: 8,
};

//
================================================================================
========
// 2. CORE VALIDATION FUNCTIONS
//
================================================================================
========

/**
 * RULE 1: Phone number pattern validation
 * @param {string} phone - Input phone number
 * @returns {{ isValid: boolean, message: string }}
 */
function validatePhoneNumber(phone) {
  if (!phone || typeof phone !== 'string') {
    return {
      isValid: false,
      rule: 'RULE_1_PHONE_PATTERN',
      message: 'Phone number is required.'
    };
  }

  // Strip common visual formatting characters (spaces, dashes, parentheses)
  const cleanPhone = phone.replace(/[-.\s()]/g, '');

  if (cleanPhone.length === 0) {
    return {
      isValid: false,
      rule: 'RULE_5_REQUIRED',
      message: 'Phone number is required.'
    };
  }

  if (!VALIDATION_PATTERNS.PHONE.test(cleanPhone)) {
    return {
```

```
    isValid: false,
    rule: 'RULE_1_PHONE_PATTERN',
    message: 'Phone number must be a valid 10 to 14 digit number (e.g. +1234567890 or
9876543210).'
   };
 }

 return {
  isValid: true,
  rule: 'RULE_1_PHONE_PATTERN',
  message: ''
 };
}

/**
 * RULE 2: Password pattern validation
 * Requires at least: 1 uppercase, 1 lowercase, 1 number, and 1 special character.
 * @param {string} password - Input password
 * @returns {{ isValid: boolean, message: string }}
 */
function validatePasswordPattern(password) {
 if (!password) {
  return {
   isValid: false,
   rule: 'RULE_2_PASSWORD_PATTERN',
   message: 'Password pattern verification required.'
  };
 }

 if (!VALIDATION_PATTERNS.PASSWORD_UPPERCASE.test(password)) {
  return {
   isValid: false,
   rule: 'RULE_2_PASSWORD_PATTERN',
   message: 'Password requires at least one uppercase letter (A-Z).'
  };
 }

 if (!VALIDATION_PATTERNS.PASSWORD_LOWERCASE.test(password)) {
  return {
   isValid: false,
   rule: 'RULE_2_PASSWORD_PATTERN',
```

```
    message: 'Password requires at least one lowercase letter (a-z).'
   };
  }

  if (!VALIDATION_PATTERNS.PASSWORD_NUMBER.test(password)) {
   return {
    isValid: false,
    rule: 'RULE_2_PASSWORD_PATTERN',
    message: 'Password requires at least one number (0-9).'
   };
  }

  if (!VALIDATION_PATTERNS.PASSWORD_SPECIAL.test(password)) {
   return {
    isValid: false,
    rule: 'RULE_2_PASSWORD_PATTERN',
    message: 'Password requires at least one special character (!@#$%^&*).'
   };
  }

  return {
   isValid: true,
   rule: 'RULE_2_PASSWORD_PATTERN',
   message: ''
  };
}

/**
 * RULE 3: Email pattern validation
 * @param {string} email - Input email address
 * @returns {{ isValid: boolean, message: string }}
 */
function validateEmail(email) {
  if (!email || typeof email !== 'string') {
   return {
    isValid: false,
    rule: 'RULE_3_EMAIL_PATTERN',
    message: 'Email address is required.'
   };
  }
```

```
  const trimmedEmail = email.trim();

  if (trimmedEmail.length === 0) {
    return {
      isValid: false,
      rule: 'RULE_5_REQUIRED',
      message: 'Email address is required.'
    };
  }

  if (!VALIDATION_PATTERNS.EMAIL.test(trimmedEmail)) {
    return {
      isValid: false,
      rule: 'RULE_3_EMAIL_PATTERN',
      message: 'Please enter a valid email address format (e.g. name@domain.com).'
    };
  }

  return {
    isValid: true,
    rule: 'RULE_3_EMAIL_PATTERN',
    message: ''
  };
}

/**
 * RULE 4: Password length validation
 * @param {string} password - Input password
 * @param {number} minLength - Minimum length threshold (default: 8)
 * @returns {{ isValid: boolean, message: string }}
 */
function validatePasswordLength(password, minLength =
VALIDATION_PATTERNS.PASSWORD_MIN_LENGTH) {
  if (!password || typeof password !== 'string') {
    return {
      isValid: false,
      rule: 'RULE_4_PASSWORD_LENGTH',
      message: 'Password is required.'
    };
  }
```

```
  if (password.length < minLength) {
    return {
      isValid: false,
      rule: 'RULE_4_PASSWORD_LENGTH',
      message: `Password length must be at least ${minLength} characters.`
    };
  }

  if (password.length > 64) {
    return {
      isValid: false,
      rule: 'RULE_4_PASSWORD_LENGTH',
      message: 'Password length cannot exceed 64 characters.'
    };
  }

  return {
    isValid: true,
    rule: 'RULE_4_PASSWORD_LENGTH',
    message: ''
  };
}

/**
 * RULE 5: Required field validation (Non-empty verification)
 * @param {string} value - Value of field
 * @param {string} fieldName - Human readable name of field
 * @returns {{ isValid: boolean, message: string }}
 */
function validateRequired(value, fieldName = 'This field') {
  if (value === undefined || value === null) {
    return {
      isValid: false,
      rule: 'RULE_5_REQUIRED',
      message: `${fieldName} is required.`
    };
  }

  const stringVal = String(value).trim();

  if (stringVal.length === 0) {
```

```
    return {
      isValid: false,
      rule: 'RULE_5_REQUIRED',
      message: `${fieldName} is required.`
    };
  }

  return {
    isValid: true,
    rule: 'RULE_5_REQUIRED',
    message: ''
  };
}

/**
 * HELPER: Password confirmation match validation
 * @param {string} password - Primary password
 * @param {string} confirmPassword - Confirmation password
 * @returns {{ isValid: boolean, message: string }}
 */
function validatePasswordMatch(password, confirmPassword) {
  const reqCheck = validateRequired(confirmPassword, 'Password confirmation');
  if (!reqCheck.isValid) {
    return reqCheck;
  }

  if (password !== confirmPassword) {
    return {
      isValid: false,
      rule: 'PASSWORD_MISMATCH',
      message: 'Passwords do not match.'
    };
  }

  return {
    isValid: true,
    rule: 'PASSWORD_MISMATCH',
    message: ''
  };
}
```

```
//
================================================================================
// 3. COMPREHENSIVE FORM VALIDATOR
//
================================================================================

/**
 * Validates an entire authentication submission object
 * @param {Object} data - Form data fields
 * @param {string} data.email - User email
 * @param {string} data.password - User password
 * @param {string} [data.name] - User full name (Sign-up only)
 * @param {string} [data.phone] - User phone number (Sign-up only)
 * @param {string} [data.confirmPassword] - Confirm password (Sign-up only)
 * @param {string} [data.adminKey] - Admin security authorization key
 * @param {boolean} isSignUp - True if registration mode, False if login mode
 * @param {string} portal - 'client' or 'admin'
 * @returns {{ isValid: boolean, errors: Object }}
 */
function validateAuthForm(data, isSignUp = false, portal = 'client') {
  const errors = {};

  // 1. Email Validation (Rule 3 & 5)
  const emailRes = validateEmail(data.email);
  if (!emailRes.isValid) {
    errors.email = emailRes.message;
  }

  // 2. Password Length & Required (Rule 4 & 5)
  const passLengthRes = validatePasswordLength(data.password, 8);
  if (!passLengthRes.isValid) {
    errors.password = passLengthRes.message;
  } else if (isSignUp) {
    // 3. Password Pattern (Rule 2) - Enforced during Registration
    const passPatternRes = validatePasswordPattern(data.password);
    if (!passPatternRes.isValid) {
      errors.password = passPatternRes.message;
    }
  }
```

```
  // 4. Registration-Only Validations
  if (isSignUp) {
    // Rule 5: Full Name Required
    const nameRes = validateRequired(data.name, 'Full Name');
    if (!nameRes.isValid) {
      errors.name = nameRes.message;
    }

    // Rule 1 & 5: Phone Number Pattern & Required
    const phoneRes = validatePhoneNumber(data.phone);
    if (!phoneRes.isValid) {
      errors.phone = phoneRes.message;
    }

    // Confirm Password Match
    const matchRes = validatePasswordMatch(data.password, data.confirmPassword);
    if (!matchRes.isValid) {
      errors.confirmPassword = matchRes.message;
    }

    // Admin Security Key Check
    if (portal === 'admin') {
      const adminKeyRes = validateRequired(data.adminKey, 'Admin Authorization Security
Key');
      if (!adminKeyRes.isValid) {
        errors.adminKey = adminKeyRes.message;
      }
    }
  }

  return {
    isValid: Object.keys(errors).length === 0,
    errors: errors
  };
}

//
================================================================================
========
// 4. UI INTERACTIVE VALIDATION BINDINGS (For index.html Modal)
```

```
//
========================================================================
========

/**
 * Live password strength indicators
 * @param {string} password - Live password text
 */
function updatePasswordMeter(password) {
  const val = password || '';
  const lenEl = document.getElementById('pw-rule-len');
  const upperEl = document.getElementById('pw-rule-upper');
  const lowerEl = document.getElementById('pw-rule-lower');
  const symEl = document.getElementById('pw-rule-sym');

  if (lenEl) {
    lenEl.className = val.length >= 8 ? 'text-emerald-600 font-semibold' : 'text-neutral-400';
  }
  if (upperEl) {
    upperEl.className = VALIDATION_PATTERNS.PASSWORD_UPPERCASE.test(val) ? 'text-
emerald-600 font-semibold' : 'text-neutral-400';
  }
  if (lowerEl) {
    lowerEl.className = VALIDATION_PATTERNS.PASSWORD_LOWERCASE.test(val) ? 'text-
emerald-600 font-semibold' : 'text-neutral-400';
  }
  if (symEl) {
    const hasNumAndSym = VALIDATION_PATTERNS.PASSWORD_NUMBER.test(val) &&
VALIDATION_PATTERNS.PASSWORD_SPECIAL.test(val);
    symEl.className = hasNumAndSym ? 'text-emerald-600 font-semibold' : 'text-neutral-
400';
  }
}

/**
 * DOM Helper to highlight invalid input fields and display inline error text
 */
function setFieldValidationUI(inputId, errorId, errorMsg) {
  const inputEl = document.getElementById(inputId);
  const errorEl = document.getElementById(errorId);
```

```
  if (inputEl) {
    if (errorMsg) {
      inputEl.classList.add('border-rose-500', 'bg-rose-50/20');
      inputEl.classList.remove('border-neutral-300', 'dark:border-neutral-700');
    } else {
      inputEl.classList.remove('border-rose-500', 'bg-rose-50/20');
      inputEl.classList.add('border-neutral-300', 'dark:border-neutral-700');
    }
  }

  if (errorEl) {
    if (errorMsg) {
      errorEl.textContent = errorMsg;
      errorEl.classList.remove('hidden');
    } else {
      errorEl.textContent = '';
      errorEl.classList.add('hidden');
    }
  }
}

/**
 * Main form submission handler applying all 5 JS validation checks
 * @param {Event} e - Form submit event
 */
function handleAuthSubmit(e) {
  if (e && e.preventDefault) {
    e.preventDefault();
  }

  const emailVal = (document.getElementById('auth-email') || {}).value || '';
  const passwordVal = (document.getElementById('auth-password') || {}).value || '';
  const nameVal = (document.getElementById('auth-name') || {}).value || '';
  const phoneVal = (document.getElementById('auth-phone') || {}).value || '';
  const confirmVal = (document.getElementById('auth-confirm') || {}).value || '';
  const adminKeyVal = (document.getElementById('auth-adminkey') || {}).value || '';

  const isSignUp = typeof isSignUpMode !== 'undefined' ? isSignUpMode : false;
  const portal = typeof currentAuthPortal !== 'undefined' ? currentAuthPortal : 'client';

  const formData = {
```

```
  email: emailVal,
  password: passwordVal,
  name: nameVal,
  phone: phoneVal,
  confirmPassword: confirmVal,
  adminKey: adminKeyVal
 };

 // Run the validation engine
 const result = validateAuthForm(formData, isSignUp, portal);

 // Reflect errors in UI
 setFieldValidationUI('auth-email', 'err-email', result.errors.email || '');
 setFieldValidationUI('auth-password', 'err-password', result.errors.password || '');

 if (isSignUp) {
  setFieldValidationUI('auth-name', 'err-name', result.errors.name || '');
  setFieldValidationUI('auth-phone', 'err-phone', result.errors.phone || '');
  setFieldValidationUI('auth-confirm', 'err-confirm', result.errors.confirmPassword || '');
  if (portal === 'admin') {
   setFieldValidationUI('auth-adminkey', 'err-adminkey', result.errors.adminKey || '');
  }
 }

 const alertBox = document.getElementById('modal-alert-box');
 if (alertBox) {
  if (!result.isValid) {
   alertBox.className = 'mb-4 p-3 rounded text-xs bg-rose-50 dark:bg-rose-950/40
border border-rose-200 dark:border-rose-800 text-rose-800 dark:text-rose-200';
   alertBox.textContent = 'Validation Error: Please resolve all highlighted required fields
and patterns according to 01CE1306 rules.';
   alertBox.classList.remove('hidden');
  } else {
   alertBox.className = 'mb-4 p-3 rounded text-xs bg-emerald-50 dark:bg-emerald-
950/40 border border-emerald-200 dark:border-emerald-800 text-emerald-800 dark:text-
emerald-200 font-medium';
   alertBox.textContent = `All 5 JavaScript validations passed! Session authenticated to
${portal === 'admin' ? 'Admin Portal' : 'Researcher Workspace'}.`;
   alertBox.classList.remove('hidden');

   // Seamless portal transition
```

```
    setTimeout(() => {
      if (typeof closeAuthModal === 'function') {
        closeAuthModal();
      }
      if (portal === 'admin' && typeof currentAppMode !== 'undefined' && currentAppMode
!== 'admin') {
        if (typeof toggleAppMode === 'function') toggleAppMode();
      } else if (portal === 'client' && typeof currentAppMode !== 'undefined' &&
currentAppMode !== 'client') {
        if (typeof toggleAppMode === 'function') toggleAppMode();
      }
    }, 1000);
   }
  }

  return result.isValid;
}

/**
 * Quick autofill credentials for demonstration and testing
 * @param {'researcher' | 'admin' | 'signup'} type
 */
function autofillAuth(type) {
  if (typeof syncAuthModalUI !== 'function') return;

  if (type === 'researcher') {
    currentAuthPortal = 'client';
    isSignUpMode = false;
    syncAuthModalUI();
    const emailEl = document.getElementById('auth-email');
    const passEl = document.getElementById('auth-password');
    if (emailEl) emailEl.value = 'evelyn.vance@nexus.ai';
    if (passEl) passEl.value = 'Nexus@Research2026!';
  } else if (type === 'admin') {
    currentAuthPortal = 'admin';
    isSignUpMode = false;
    syncAuthModalUI();
    const emailEl = document.getElementById('auth-email');
    const passEl = document.getElementById('auth-password');
    if (emailEl) emailEl.value = 'admin@nexus.ai';
    if (passEl) passEl.value = 'Admin#NexusPass88';
```

```
  } else if (type === 'signup') {
    isSignUpMode = true;
    syncAuthModalUI();
    const nameEl = document.getElementById('auth-name');
    const phoneEl = document.getElementById('auth-phone');
    const emailEl = document.getElementById('auth-email');
    const passEl = document.getElementById('auth-password');
    const confirmEl = document.getElementById('auth-confirm');
    const keyEl = document.getElementById('auth-adminkey');

    if (nameEl) nameEl.value = 'Dr. Nirav Nimavat';
    if (phoneEl) phoneEl.value = '+19876543210';
    if (emailEl) emailEl.value = 'nirav.nimavat@nexus.ai';
    if (passEl) passEl.value = 'NexusSecure#2026';
    if (confirmEl) confirmEl.value = 'NexusSecure#2026';
    if (keyEl && currentAuthPortal === 'admin') keyEl.value = 'NX-ROOT-92610';

    updatePasswordMeter('NexusSecure#2026');
  }
}

// Export for module systems if run in Node/ES environment
if (typeof module !== 'undefined' && module.exports) {
  module.exports = {
    VALIDATION_PATTERNS,
    validatePhoneNumber,
    validatePasswordPattern,
    validateEmail,
    validatePasswordLength,
    validateRequired,
    validatePasswordMatch,
    validateAuthForm,
    handleAuthSubmit
  };
}
```